

Here is a simple example where we ask for a number from the user and attempt to convert it to an integer:

```
age = input('Please enter your age: ')
try: # attempt to convert to integer
    age = int(age)
except ValueError: # if an exception is raised trying to convert
    print('Sorry, that was not a valid number')
```

Calls to the Python Standard Library can generate standard exceptions such as `TypeError`, `ValueError`, `NameError`, `ZeroDivisionError`, and so on. In this example, if the user enters letters or a floating-point number, the built-in `int()` function raises a `ValueError` exception, and control is transferred to the code in the `except` block.

The raise Statement and Custom Exceptions

If your code detects a runtime error condition, you can use the `raise` statement to signal an exception. There are many forms of the `raise` statement, but the standard approach is to use this syntax:

```
raise <ExceptionName>('<Any string to describe the error>')
```

For the `<ExceptionName>`, you have three options. First, if there is a standard exception that matches the error you have detected (`TypeError`, `ValueError`, `NameError`, `ZeroDivisionError`, and so on), it's fine to use that. You can also add your own description string:

```
raise ValueError('You need to specify an integer')
```

Second, you can use the generic `Exception` exception:

```
raise Exception('The amount cannot be a floating-point number')
```

However, this is generally frowned upon because the standard practice is to write `except` statements to look for exceptions by name, and this does not provide a specific name.